# VISVESVARAYA TECHNOLOGICAL UNIVERSITY

**"JnanaSangama", Belgaum -590014, Karnataka.**

**LAB REPORT on**

# Machine Learning (23CS6PCMAL)

***Submitted by***

**D A Chethan (1BM23CS083)**

***in partial fulfillment for the award of the degree of***

**BACHELOR OF ENGINEERING**

***in***

**COMPUTER SCIENCE AND ENGINEERING**

**B.M.S. COLLEGE OF ENGINEERING**

**(Autonomous Institution under VTU)**

**BENGALURU-560019**

**B.M.S. College of Engineering,**
**Bull Temple Road, Bangalore 560019**
(Affiliated To Visvesvaraya Technological University, Belgaum)
**Department of Computer Science and Engineering**

## CERTIFICATE

This is to certify that the Lab work entitled "Machine Learning (23CS6PCMAL)" carried out by **D A Chethan (1BM23CS083),** who is bonafide student of **B.M.S. College of Engineering.** It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum. The Lab report has been approved as it satisfies the academic requirements in respect of an Machine Learning (23CS6PCMAL) work prescribed for the said degree.

| | |
|---|---|
| Saritha A. N.<br>Assistant Professor<br>Department of CSE, BMSCE | Dr. Kavitha Sooda<br>Professor & HOD<br>Department of CSE, BMSCE |

# Index

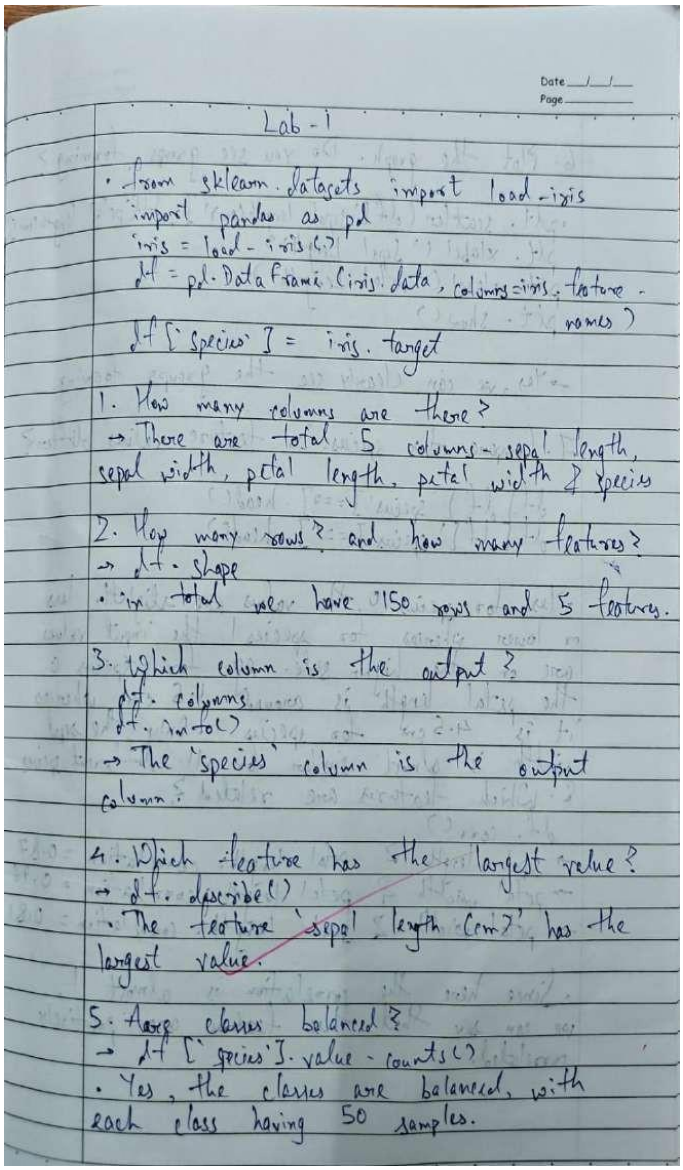

| Sl. No. | Date | Experiment Title | Page No. |
|---|---|---|---|
| 1 | 4-2-2026 | Write a python program to import and export data using Pandas library functions | 4-7 |
| 2 | 11-2-2026 | Demonstrate various data pre-processing techniques for a given dataset | 8-13 |
| 3 | 25-2-2026 | Implement Linear and Multi-Linear Regression algorithm using appropriate dataset | 14-18 |
| 4 | 25-2-2026 | Build Logistic Regression Model for a given dataset | 19-23 |
| 5 | 11-3-2026 | Use an appropriate data set for building the decision tree (ID3) and apply this knowledge to classify a new sample. | 24-28 |
| 6 | 25-3-2026 | Build KNN Classification model for a given dataset. | 29-31 |
| 7 | 1-4-2026 | Build Support vector machine model for a given dataset | 32-34 |
| 8 | 1-4-2026 | Implement Random forest ensemble method on a given dataset. | 35-37 |
| 9 | 1-4-2026 | Implement Boosting ensemble method on a given dataset. | 38-41 |
| 10 | 8-4-2026 | Build k-Means algorithm to cluster a set of data stored in a .CSV file. | 42-45 |
| 11 | 8-4-2026 | Implement Dimensionality reduction using Principal Component Analysis (PCA) method. | 46-48 |

Github Link:
https://github.com/DAChethan2485/ML/tree/main

## **Program 1**

Write a python program to import and export data using Pandas library functions

Screenshot:



Date __/__/__
Page _____

Lab - 1

• from sklearn.datasets import load_iris
import pandas as pd
iris = load_iris()
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df['species'] = iris.target

1. How many columns are there?
→ There are total 5 columns - sepal length, sepal width, petal length, petal width & species

2. How many rows? and how many features?
→ df.shape
• in total we have 150 rows and 5 features.

3. Which column is the output?
df.columns
df.info()
→ The 'species' column is the output column.

4. Which feature has the largest value?
→ df.describe()
• The feature 'sepal length (cm)' has the largest value.

5. Are classes balanced?
→ df['species'].value_counts()
• Yes, the classes are balanced, with each class having 50 samples.

Date __/__/__
Page ____

6. Plot the graph. Do you see groups forming?

```
plt.scatter(df['sepal length(cm)'], df['petal length(cm)'])
plt.xlabel('Sepal Length')
plt.ylabel('Petal Length')
plt.show()
```

→ Yes, we can clearly see the groups forming.

7. Compare two species. Do feature values differ?

```
df[df['species'] == 0].head()
df[df['species'] == 1].head()
```

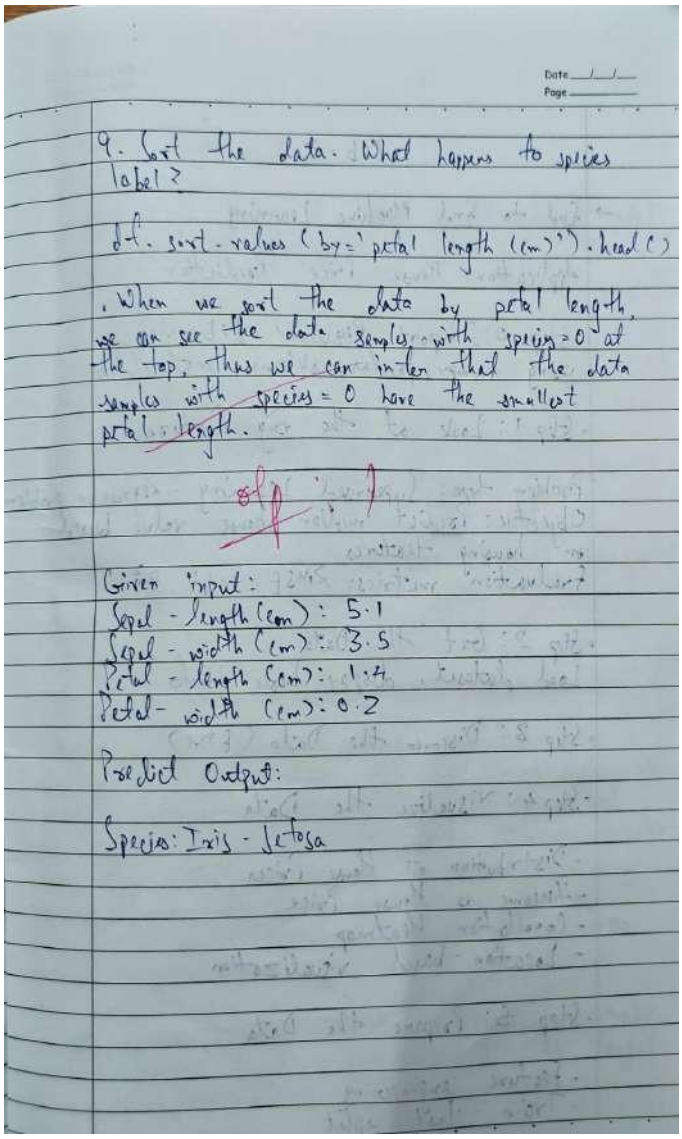
- Yes, for species 0 the values are slightly less or lower whereas for species 1 the input values are on the higher side. eg: for species 0 the petal length is around 1.5 cm whereas it is 4.5 cm for species 1. Only the sepal width is almost similar for all different species.

8. Which features are related?

```
df.corr()
```

→ sepal length & petal length: correlation = 0.87
→ petal width & petal length: correlation = 0.96
→ petal width & sepal length: correlation = 0.81

- Since here the correlation is almost 1, we can say that these features are positively correlated.



Date __/__/__
Page _____

9. Sort the data. What happens to species label?

df.sort_values(by='petal length (cm)').head()

. When we sort the data by petal length, we can see the data samples with species = 0 at the top. Thus we can infer that the data samples with species = 0 have the smallest petal length.

Given input:
Sepal - length (cm): 5.1
Sepal - width (cm): 3.5
Petal - length (cm): 1.4
Petal - width (cm): 0.2

Predict Output:

Species: Iris - Setosa

Code:

```
from sklearn.datasets import load_iris

import pandas as pd

iris = load_iris() df = pd.DataFrame(iris.data,

columns=iris.feature_names) df['species'] = iris.target

df.head() df.shape


df.columns df.info df.describe()

df['species'].value_counts() import matplotlib.pyplot
```

```
as plt plt.scatter(df['sepal length (cm)'],df['petal length
(cm)']) plt.xlabel("Sepal Length") plt.ylabel("Petal
Length") plt.show() df[df['species']==0].head()
df[df['species']==1].head() df.corr()
df.sort_values(by='petal length (cm)').head()
```

## Program 2

Demonstrate various data pre-processing techniques for a given dataset

Screenshot:

Date __/__/__
Page ____

# Lab - 2

→ End to End Machine Learning

Application: House Price Prediction

- Step 0: Import Required Libraries
using numpy, pandas, sklearn, matplotlib.

- Step 1: Look at the Big Picture

Problem type: Supervised Learning - Regression problem
Objective: Predict median house value based on housing features
Evaluation metrics: RMSE, MAE, $R^2$

- Step 2: Get the Data
Load dataset, display basic info

- Step 3: Discover the Data (EDA)

- Step 4: Visualize the Data

- Distribution of House Prices
- Income vs House Price
- Correlation Heatmap
- Location-based visualization

- Step 5: Prepare the Data

- Feature engineering
- Train-test split

Date __/__/__
Page _____

- Feature scaling

• Step 6: Select & Train a Baseline Model
- use Linear Regression & find out the error

• Step 7: Train Advanced Models.

- Decision Tree Regressor
- Random Forest Regressor

• Step 8: Fine-Tune the model

- Hyperparameter Tuning for Random Forest model

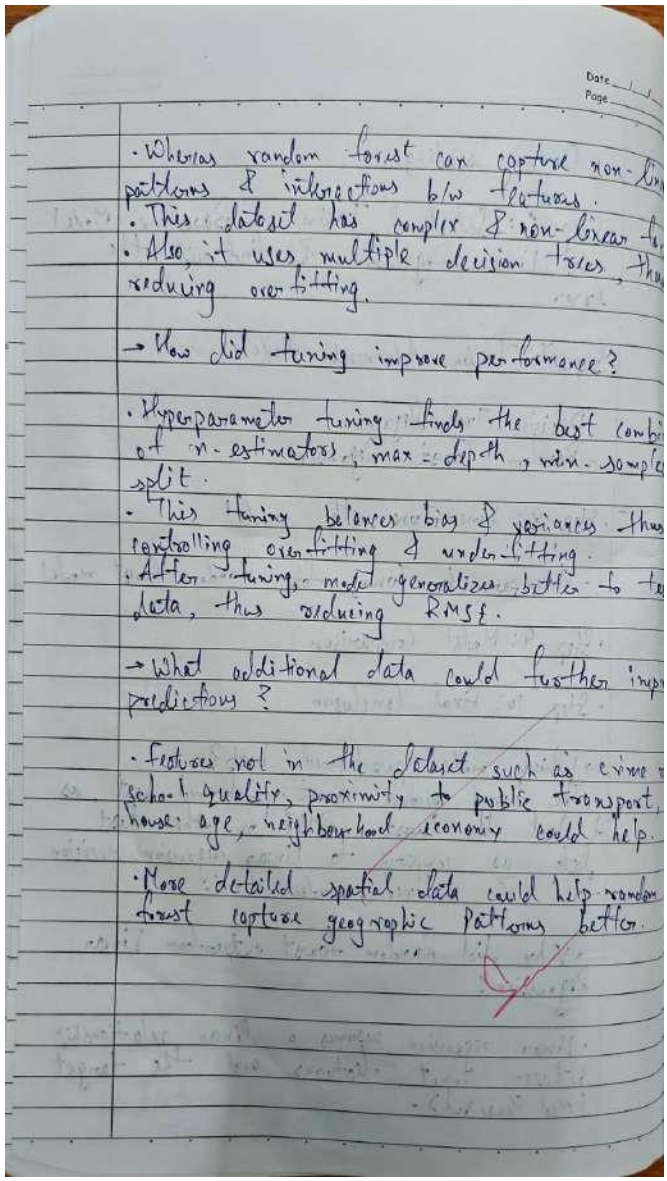
• Step 9: Model Comparison

• Step 10: Final Conclusion

→ Which model performed best?
The tuned random forest performed best, as it had the lowest RMSE on the test data as compared to Linear regression, decision tree & untuned random forest.

→ Why did Random forest outperform Linear Regression?

• Linear regression assumes a linear relationship between input features and the target (Median House Val).



Date
Page

- Whereas random forest can capture non-lin patterns & interactions b/w features.
- This dataset has complex & non-linear fe
- Also, it uses multiple decision trees, thus reducing overfitting.

→ How did tuning improve performance?

- Hyperparameter tuning finds the best combi of n-estimators, max-depth, min-sample split.
- This tuning balances bias & variances thus controlling overfitting & underfitting.
- After tuning, model generalizes better to te data, thus reducing RMSE.

→ What additional data could further impr predictions?

- Features not in the dataset such as crime r school quality, proximity to public transport, house age, neighbourhood economy could help.
- More detailed spatial data could help random forest capture geographic patterns better.

Code:

import numpy as np import pandas as pd import matplotlib.pyplot as plt import seaborn as sns from sklearn.datasets import fetch_california_housing from sklearn.model_selection import train_test_split, GridSearchCV from sklearn.preprocessing import StandardScaler

```
from sklearn.linear_model import LinearRegression from sklearn.tree import
DecisionTreeRegressor from sklearn.ensemble import
RandomForestRegressor from sklearn.metrics import mean_squared_error,
mean_absolute_error,r2_score housing =
fetch_california_housing(as_frame=True) df = housing.frame df.head()
df.shape df.info() df.head() df.describe() df.isnull().sum() plt.figure()
sns.histplot(df['MedHouseVal'], bins=30, kde=True) plt.xlabel('Median House
Value') plt.ylabel('Frequency') plt.title('Distribution of House Prices')
plt.show() plt.figure() sns.scatterplot(x=df['MedInc'], y=df['MedHouseVal'],
alpha=0.5) plt.xlabel("Median Income") plt.ylabel("Median House Value")
plt.title("Income vs House Price") plt.show()
plt.figure(figsize=(10, 8)) sns.heatmap(df.corr(),
annot=True, cmap='coolwarm') plt.title("Feature
Correlation Heatmap") plt.show() plt.figure()
plt.scatter(     df['Longitude'],     df['Latitude'],
c=df['MedHouseVal'],     cmap='viridis',     s=5
)
plt.colorbar(label="House Value")
plt.xlabel("Longitude")
plt.ylabel("Latitude")
plt.title("House Prices by Location")
plt.show()
X = df.drop('MedHouseVal',axis = 1)
y = df['MedHouseVal']
```

```
X_train,X_test,y_train,y_test =
train_test_split(X,y,test_size=0.2,random_state=42) scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
linreg = LinearRegression()
linreg.fit(X_train_scaled,y_train) y_pred_lr =
linreg.predict(X_test_scaled) rmse_lr =
np.sqrt(mean_squared_error(y_test,
y_pred_lr)) mae_lr =
mean_absolute_error(y_test, y_pred_lr) r2_lr
= r2_score(y_test, y_pred_lr) rmse_lr, mae_lr,
r2_lr dt =
DecisionTreeRegressor(random_state=42)
dt.fit(X_train, y_train) y_pred_dt =
dt.predict(X_test) rmse_dt =
np.sqrt(mean_squared_error(y_test,
y_pred_dt)) rmse_dt rf =
RandomForestRegressor(random_state=42)
rf.fit(X_train, y_train) y_pred_rf = rf.predict(X_test)
rmse_rf = np.sqrt(mean_squared_error(y_test,
y_pred_rf)) rmse_rf
```

**Program 3**

Implement Linear and Multi-Linear Regression algorithm using appropriate dataset Screenshot:

training labels = [ 'fail', 'fail', 'fail', 'Pass', 'Pass', 'Pass', 'Pass' ]

- Given input = [2.5, 55] and k = 3
- Predicted Output ⇒ fail
- Given input = [5.5, 87]
- Predicted Output ⇒ Pass

→ Multiple Linear Regression

- Here we have a single output (dependent) feature and multiple input (independent) features in our dataset.

$$y = b_0 + b_1 x_1 + b_2 x_2 + \dots + b_n x_n$$

$$b = (X^T X)^{-1} X^T y$$

- Algorithm:

i> Add bias columns (a column of 1s)

ii> Apply Normal Equation

iii> Store the found out coefficients

iv> Use the regression equation to predict

Date __/__/__
Page _____

values of the new data point.

→ Example

- Inputs - training data:

[Study Days Per Week, Attendance Percentage]

= [ [1, 60], [2, 65], [3, 70], [4, 75], [5, 80]
[6, 85], [7, 90], [8, 92], [9, 95], [10, 98] ]

- Output (training data): [Percentage scored]

[40, 45, 50, 55, 60, 65, 70, 78, 85, 92]

- Give input: [6, 88]
- Predicted Exam Percentage: 62.0

Code:

```
import numpy as np import pandas as pd from sklearn.datasets import load_iris,
load_breast_cancer, fetch_california_housing from sklearn.model_selection
import train_test_split from sklearn.tree import DecisionTreeClassifier,
DecisionTreeRegressor from sklearn.metrics import accuracy_score,
precision_score, recall_score, f1_score from sklearn.metrics import
confusion_matrix from sklearn.metrics import mean_absolute_error,
mean_squared_error, r2_score def run_classification(dataset_loader, name):
    data =
dataset_loader()     X =
data.data     y =
data.target
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=42
    )
    model = DecisionTreeClassifier(criterion="entropy",
random_state=42)     model.fit(X_train, y_train)     predictions =
model.predict(X_test)     print("\n===== Classification Dataset:", name,
"=====")     accuracy = accuracy_score(y_test, predictions)     precision
= precision_score(y_test, predictions, average="weighted")     recall =
recall_score(y_test, predictions, average="weighted")     f1 =
f1_score(y_test, predictions, average="weighted")

    cm = confusion_matrix(y_test,
predictions)     print("Accuracy:", accuracy)
```

```
    print("Precision:", precision)
    print("Recall:", recall)     print("F1 Score:",
    f1)     print("Confusion Matrix:\n", cm) def
    run_regression():     data =
    fetch_california_housing()     X = data.data
    y = data.target
        X_train, X_test, y_train, y_test = train_test_split(
            X, y, test_size=0.2, random_state=42
        )
        model = DecisionTreeRegressor(random_state=42)
    model.fit(X_train, y_train)     predictions = model.predict(X_test)
    print("\n===== Regression Dataset: California Housing =====")
    mae = mean_absolute_error(y_test, predictions)     mse =
    mean_squared_error(y_test, predictions)     rmse = np.sqrt(mse)
    r2 = r2_score(y_test, predictions)     print("MAE:", mae)
    print("MSE:", mse)     print("RMSE:", rmse)

        print("R2 Score:", r2) if
    __name__ == "__main__":
    run_classification(load_iris,                "Iris                Dataset")
run_classification(load_breast_cancer,    "Breast    Cancer    Dataset")
run_regression()
```

**<u>Program 4</u>** Build

Logistic Regression Model for a given dataset  Screenshot:

Date 25, 3, 26
Page

# Lab-6

→ Logistic Regression

- Binary Logistic Regression (2 classes)
The main goal here is to use a sigmoid function to convert linear output into probability and classify data into binary classes.

- Algorithm:
i) Initialize parameters
Weights: $w_1, w_2, \ldots, w_n$
Bias: $b$

ii) Linear combination
$$z = w_1 x_1 + w_2 x_2 + \ldots + w_n x_n + b$$

iii) Sigmoid function
$$\sigma(z) = \frac{1}{1+e^{-z}}$$

iv) Compute loss: Binary cross-entropy (Log Loss)
$$J = \frac{-1}{m} \sum \left[ y \log(\hat{y}) + (1-y) \log(1-\hat{y}) \right]$$

v) Update parameters
- Use gradient descent
$$w = w - \alpha \cdot \frac{\partial J}{\partial w}$$

Date __/__/__
Page ____

vi> Repeat until convergence

vii> Prediction Rule
if $y \geq 0.5 \rightarrow$ class 1
else $\rightarrow$ class 0

- Multiclass Logistic Regression
→ goal here is to take the input data and predict a class (1, 2, 3 ..., k) using softmax regression.

i> Initialize parameters
separate weights for each class: $w^{(1)}, w^{(2)}, \ldots, w^{(k)}$.

ii> Compute scores for each class
for each class k
$$z_k = w_k \cdot x + b_k$$

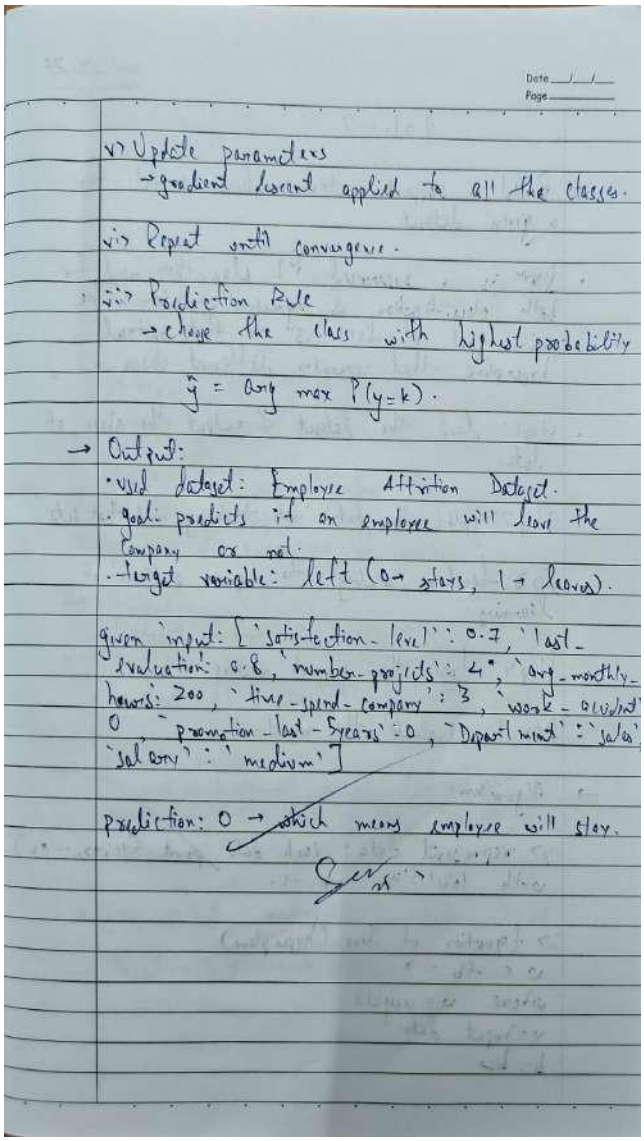
iii> Apply softmax function

$$P(y=k) = \frac{e^{z_k}}{\sum_{j=1}^{k} e^{z_j}}$$

- Convert all scores into probabilities such that all probability sum = 1.

iv> Compute loss
categorical cross entropy:

$$J = -\sum y_k \log(\hat{y}_k)$$



v) Update parameters
→ gradient descent applied to all the classes.

vi) Repeat until convergence.

vii) Prediction Rule
→ choose the class with highest probability

$$\hat{y} = \arg\max P(y=k).$$

→ Output:
· used dataset: Employee Attrition Dataset.
· goal: predicts if an employee will leave the company or not.
· target variable: left (0 → stays, 1 → leaves).

given input: ['satisfaction_level': 0.7, 'last_evaluation': 0.8, 'number_projects': 4, 'avg_monthly_hours': 200, 'time_spend_company': 3, 'work_accident': 0, 'promotion_last_5years': 0, 'Department': 'sales', 'salary': 'medium']

prediction: 0 → which means employee will stay.

Sen

Code:

```
import numpy as np from sklearn.linear_model

import LinearRegression

X = np.array([

    [1, 60],

    [2, 65],

    [3, 70],
```

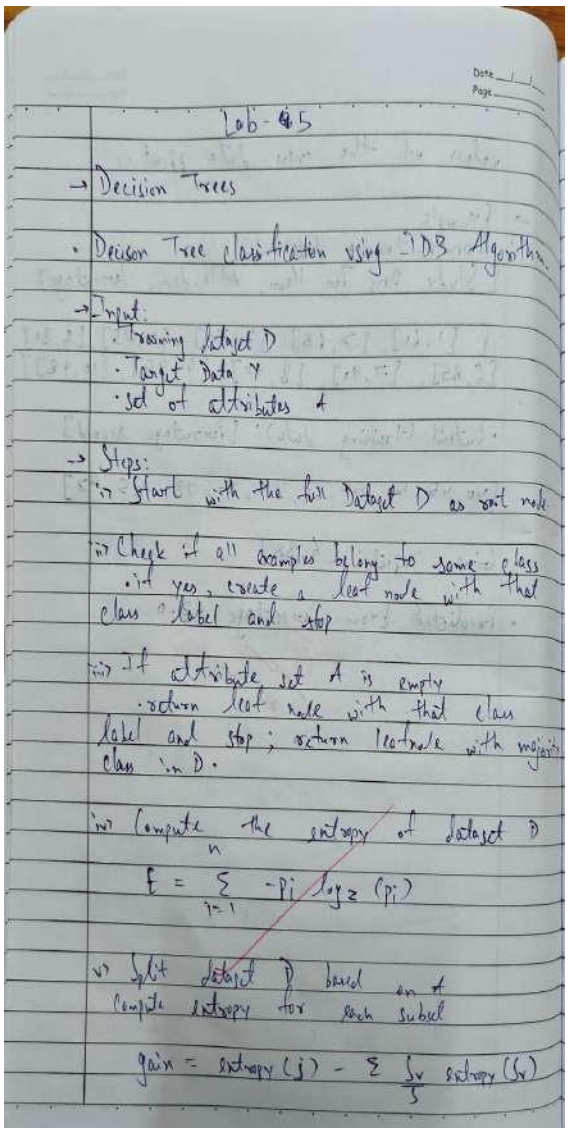
```
    [4, 75],
    [5, 80],
    [6, 85],
    [7, 90],
    [8, 92],
    [9, 95],
    [10, 98]
])
y = np.array([
    40,
    45,
    50,
    55,
    60,
    65,
    70,
    78,
    85,
    92
])
model = LinearRegression()
model.fit(X, y) new_student =
np.array([[6, 88]])
predicted_percentage =
```

```
model.predict(new_student)

print("Predicted Exam

Percentage:",

round(predicted_percentage[0],

2)) print("Intercept (b0):",

model.intercept_)

print("Coefficients (b1, b2):",

model.coef_)
```

## Program 5

Use an appropriate data set for building the decision tree (ID3) and apply this knowledge to classify a new sample.

Screenshot:



Lab - 5

→ Decision Trees

• Decision Tree classification using ID3 Algorithm

→ Input:
- Training dataset D
- Target Data Y
- set of attributes A

→ Steps:

i) Start with the full Dataset D as root node

ii) Check if all examples belong to same class
- if yes, create a leaf node with that class label and stop

iii) If attribute set A is empty
- return leaf node with that class label and stop; return leafnode with majority class in D.

iv) Compute the entropy of dataset D

$$E = \sum_{i=1}^{n} -p_i \log_2 (p_i)$$

v) Split dataset D based on A
Compute entropy for each subset

$$\text{gain} = \text{entropy}(S) - \sum_{S} \frac{S_v}{S} \text{entropy}(S_v)$$

vi) Select attribute with highest info gain.

vii) Create a DT using selected attributes

viii) Repeat until no attributes are left, dataset becomes empty.

ix) Return constructed Tree.

Output

~~Cart~~

petal length <= 2.45

no / \ yes

setosa petal width <= 1.75

no / \ yes

versicolor virginica

- Decision tree regression using CART

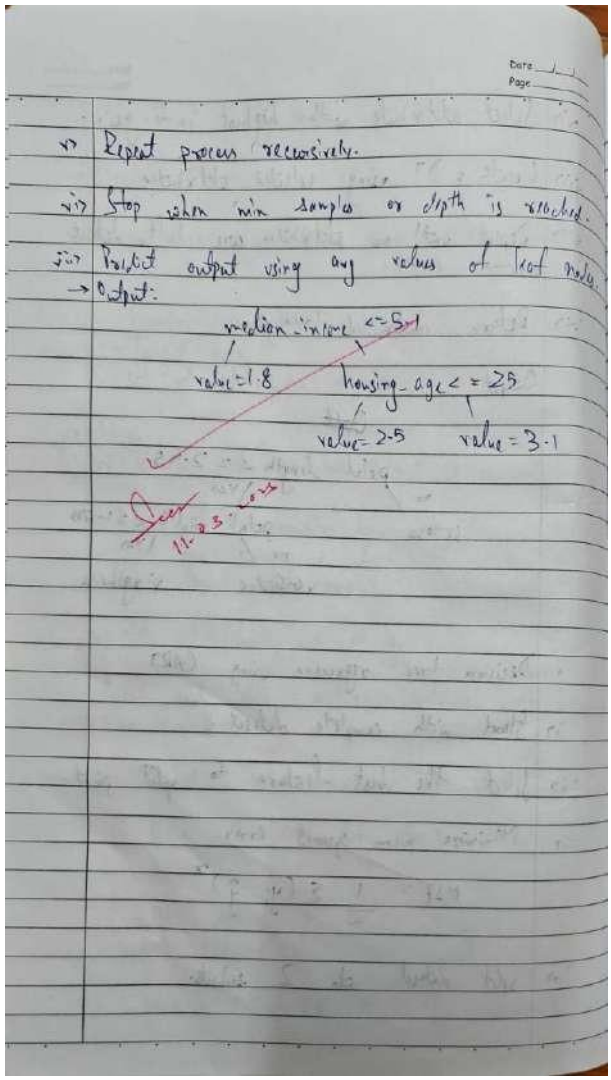
i) Start with complete dataset.

ii) Select the best feature to split point.

iii) Minimize mean squared error

$$MSE = \frac{1}{n} \Sigma (y_i - \hat{y})^2$$

iv) split dataset into 2 subsets.



v) Repeat process recursively.

vi) Stop when min samples or depth is reached.

vii) Predict output using avg values of leaf nodes.

→ Output:

median_income <= 5.1

value = 1.8

housing_age <= 25

value = 2.5

value = 3.1

Seen 11.23.2025

Code:

```
import pandas as pd from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression from
sklearn.preprocessing import StandardScaler from sklearn.metrics import
accuracy_score df = pd.read_csv("HR_comma_sep.csv") df =
pd.get_dummies(df, columns=['Department', 'salary'], drop_first=True)
X = df.drop('left', axis=1)
y = df['left']
X_train, X_test, y_train, y_test = train_test_split(
```

```
    X, y, test_size=0.2, random_state=42
)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train) X_test =
scaler.transform(X_test) model =
LogisticRegression(max_iter=1000, solver='liblinear')
model.fit(X_train, y_train y_pred = model.predict(X_test)
from sklearn.metrics import accuracy_score,
confusion_matrix print("Accuracy:", accuracy_score(y_test,
y_pred)) cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:\n", cm) input_data = {
    'satisfaction_level': 0.7,
    'last_evaluation': 0.8,
    'number_project': 4,
    'average_montly_hours': 200,
    'time_spend_company': 3,
    'Work_accident': 0,
    'promotion_last_5years': 0,
    'Department_sales': 1,
    'salary_medium': 1
} sample_df = pd.DataFrame([input_data]) sample_df =
sample_df.reindex(columns=X.columns, fill_value=0)
sample_scaled = scaler.transform(sample_df) prediction =
model.predict(sample_scaled) if prediction[0] == 1:
```

```
    print("Employee will LEAVE ")
else:
      print("Employee will STAY ")
```

## Program 6

Build KNN Classification model for a given dataset.

Screenshot:

Date __/__/__
Page _____

# Lab-3, Lab-4

→ KNN Algorithm

• To implement KNN we need 3 things: set of stored datapoints, k value, distance metric.

i) Select k: Choose the number of neighbours (k) to consider.

ii) Calculate Distance: Calculate the distance (commonly euclidean distance) between the new data point and all existing data points in the training set.

iii) Find nearest neighbours: identify the k data points with the smallest distances.

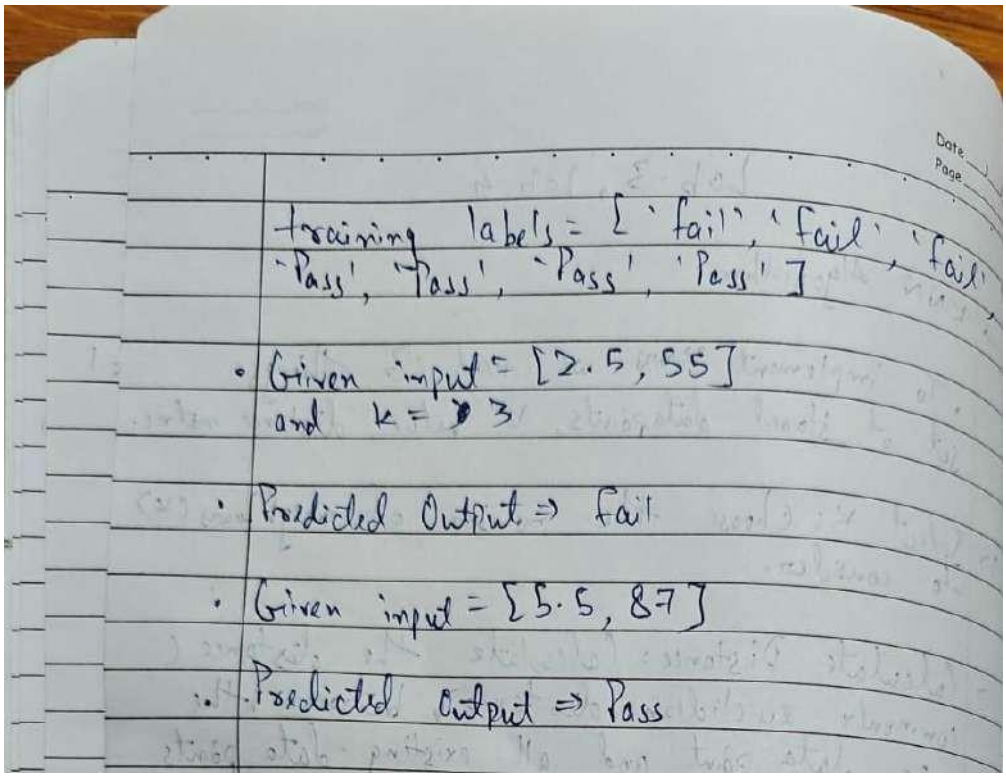
iv) Assign class/value:
- for classification: The new point is assigned the class that is most common among its k neighbors.

- for regression: The new point is assigned the average of the values of its k neighbors.

→ Example

[Study hours per day, Attendance Percentage]

training data = [ [1,40], [2,50], [1.5, 43], [3,60], [4,75], [5,85], [6,90], [7,95] ]



training labels = ['fail', 'fail', 'fail', 'Pass', 'Pass', 'Pass', 'Pass']

- Given input = [2.5, 55] and k = 3
- Predicted Output ⇒ fail
- Given input = [5.5, 87]
- Predicted Output ⇒ Pass

Code:

```
import numpy as np from sklearn.neighbors import

KNeighborsClassifier from

sklearn.model_selection import train_test_split

X = np.array([

    [1, 40],

    [2, 50],

    [1.5, 43],

    [3, 60],

    [4, 75],

    [5, 85],

    [6, 90],

    [7, 95]
```

```
]) y = np.array([0,0,0,0,1,1,1,1]) model =
KNeighborsClassifier(n_neighbors=3)
model.fit(X, y) new_student = np.array([[2.5,
55]]) prediction = model.predict(new_student)
if prediction[0] == 1:     print("Prediction:
Pass") else:     print("Prediction: Fail")
```

**Program 7** Build

Support vector machine model for a given dataset  Screenshot:

Date / / 2025
Page

# Lab - 7

→ Build a support vector machine model for a given dataset

- SVM is a supervised ML algorithm used for both classification & regression problem. The main goal of SVM is to find optimal hyperplane that seperates different classes.

- Step1: load the dataset & output the shape of data.

- Step2: split the data into training and test sets

- Step3: feature scaling: to improve pattern of learning.

- Step4: train the support vectors

- Step 5: Evaluate the model.

→ Algorithm:

i) represent data: each data point $(x_1, x_2, \ldots x_n)$ with label = +1 or -1.

ii) Equation of line (hyperplane)
$w \cdot x + b = 0$
where $w$ = weights
$x$ = input data
$b$ = bias

iii> Classification rule

if: $w \cdot x + b \geq 0 \Rightarrow$ class +1

$w \cdot x + b < 0 \Rightarrow$ class -1

iv> Margin concept

- SVM tries to maximise the margin given by: $\frac{2}{\|w\|}$

v> Main objective:

$\min \frac{1}{2} \|w\|^2$

vi> Identify support vectors: the points nearest to the hyperplane. They mainly influence the position & orientation of hyperplane.

→ Output:

- Dataset: Breast cancer dataset

input features: radius & texture

output: +1 (malignant) & -1 → benign

- Sample values:

input ⇒ [(17.99, 10.38), (20.57, 17.77), (13.54, 14.36), (12.47, 18.67)]

output ⇒ [+1, +1, -1, -1]

- given output: radius = 11.42, texture: 20.38

→ found output: -1 ⇒ benign

Code:

```
import pandas as pd from sklearn.model_selection import
train_test_split from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC from sklearn.metrics import
confusion_matrix, classification_report url =
"https://raw.githubusercontent.com/rolandmueller/titanic/main/tita
```

nic3.csv" df = pd.read_csv(url) df =

df[['pclass','sex','age','fare','survived']] df = df.dropna() df['sex'] =

df['sex'].map({'male': 0, 'female': 1}) X =

df[['pclass','sex','age','fare']] y = df['survived']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train) X_test =

scaler.transform(X_test) model = SVC(kernel='rbf')

model.fit(X_train, y_train) y_pred = model.predict(X_test)

print("Accuracy:", model.score(X_test, y_test)) print("\nConfusion

Matrix:\n", confusion_matrix(y_test, y_pred)) print("\nClassification

Report:\n", classification_report(y_test, y_pred)) **Program 8**

Implement Random Forest ensemble method on a given dataset.

Screenshot:

→ feature1 feature 2 cluster

| | | |
|---|---|---|
| -7.24 | -7.55 | 1 |
| -7.56 | -7.18 | 1 |
| -1.85 | 8.03 | 0 |
| 4.46 | 2.85 | 2 |
| -8.51 | -7.68 | 1 |

- Random forest

→ Titanic Dataset

| | pclass | sex | age | fare | survived |
|---|---|---|---|---|---|
| 0 | 1 | 1 | 29 | 211.33 | 1 |
| 1 | 1 | 0 | 6.91 | 151.55 | 1 |
| 2 | 1 | 1 | 2 | 151.55 | 0 |

→ Accuracy = 0.829

Report

| | precision | recall | f1-score | support |
|---|---|---|---|---|
| 0 | 0.79 | 0.9 | 0.84 | 144 |
| 1 | 0.85 | 0.71 | 0.77 | 118 |

Code:

import pandas as pd import numpy as np from sklearn.model_selection import train_test_split from sklearn.ensemble import RandomForestClassifier from sklearn.metrics import accuracy_score, classification_report, confusion_matrix df = pd.read_csv("https://raw.githubusercontent.com/rolandmueller/titanic/main/titanic3.csv") print("First 5

```
rows:") print(df.head()) df = df[["pclass", "sex",
"age", "fare", "survived"]] df["sex"] =
df["sex"].map({"male": 0, "female": 1}) df["age"] =
df["age"].fillna(df["age"].mean()) df["fare"] =
df["fare"].fillna(df["fare"].mean()) print("\nAfter
preprocessing:") print(df.head())
X = df.drop("survived", axis=1)
y = df["survived"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
model = RandomForestClassifier(
n_estimators=100,
max_depth=None,
random_state=42
)
model.fit(X_train, y_train) y_pred =
model.predict(X_test) print("\nAccuracy:",
accuracy_score(y_test, y_pred))
print("\nClassification Report:")
print(classification_report(y_test, y_pred))
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))
```

```
feature_importance = pd.Series(
model.feature_importances_,     index=X.columns
)
print("\nFeature Importance:")
print(feature_importance.sort_values(ascending=False))
```

**Program 9** Implement Boosting ensemble method on a given dataset.

Screenshot:

Date ___
Page ___

# Lab-8

→ Implement boosting ensemble method on a given dataset.

- Boosting is a powerful ensemble learning technique that combines multiple weak learners to create a strong learner.

→ Algorithm:

i) initialize weights: assign equal weights to all the training samples:

$$w_i = \frac{1}{n}$$

ii) for every iteration:

a) train weak learner using weighted dataset.

b) compute error:

$$\varepsilon = \sum_{i=1}^{n} w_i \cdot I$$

c) compute model weight:

$$\alpha_t = \frac{1}{2} \ln\left(\frac{1-\varepsilon}{\varepsilon}\right)$$

d) update weights:

$$w_i = w_i \cdot e^{-\alpha_t \cdot y_i \cdot h(x_i)}$$

e) normalize weights: so that sum of all weights = 1.

Date __/__/__
Page _____

iii) Final model: Combine all weak learners

$$H(x) = \text{sign}\left(\sum_{t=1}^{T} \alpha_t \cdot h_t(x)\right)$$

→ Here we have implemented the AdaBoost Algorithm on our dataset.

→ Output:

- Dataset: Customer purchase behaviour dataset.
- Features: age, income, no. of purchases, time spent, loyalty program.
- Target: purchase (0=No, 1=Yes).
- Sample values = [(25, 40000, 2, 5, 0), (35, 60000, 5, 15, 1), (23, 30000, 1, 3, 0), (40, 70000, 6, 18, 1)]
- Sample output = [0, 1, 0, 1]

→ given input: age = 30, income = 55000, purchases = 4, time spent = 12, loyalty = 1

- predicted output = 1 → customer will buy.

Code:

```
from sklearn.datasets import load_iris from sklearn.model_selection import
train_test_split from sklearn.ensemble import AdaBoostClassifier from
sklearn.metrics import accuracy_score, confusion_matrix, classification_report
import matplotlib.pyplot as plt
```

```
import pandas as pd iris =
load_iris() X = iris.data y =
iris.target feature_names =
iris.feature_names
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
model = AdaBoostClassifier(
n_estimators=50,
learning_rate=1.0,
random_state=42
)
model.fit(X_train, y_train) y_pred = model.predict(X_test) acc =
accuracy_score(y_test, y_pred) print("Accuracy:", acc)
print("\nClassification Report:") print(classification_report(y_test,
y_pred)) cm = confusion_matrix(y_test, y_pred) print("\nConfusion
Matrix:") print(cm) importance = pd.Series(model.feature_importances_,
index=feature_names) print("\nFeature Importance:")

print(importance)
importance.plot(kind='bar')
plt.title("AdaBoost                    Feature
Importance")      plt.ylabel("Importance
Score") plt.show()
```

**<u>Program 10</u>** Build k-

Means algorithm to cluster a set of data stored in a .CSV file.

Screenshot:

Date __/__/__
Page ____

# Lab - 9

- Build a k-means Algorithm to cluster a set of data clustered in a .csv file.

→ K-means clustering is an unsupervised learning algorithm used to partition n observations into k clusters where each observation belongs to the cluster with the nearest mean.

- Steps:

i) initialization: randomly select k data points from the dataset as initial cluster centers.

ii) Assignment step: assign each data point to its closest centroid, forming k clusters.

iii) Update:
recalculate the centroids of the newly formed clusters.

iv) repeat: repeat the step 2 & 3.

v) final:
the final output that we get will be clear clusters with their defined centroids.

Date __/__/__
Page _____

→ Output:

cluster labels for first 10 data points:

[1 1 0 2 1 2 0 2 0 0]

shape of cluster centroids: (3, 2)

→ cluster centroids:

| | |
|---|---|
| -0.209 | 1.164 |
| -1.09 | -1.262 |
| 1.3 | 0.09 |

→

| feature1 | feature 2 | cluster |
|---|---|---|
| -7.24 | -7.55 | 1 |
| -7.56 | -7.18 | 1 |
| -1.85 | 8.03 | 0 |
| 4.46 | 2.85 | 2 |
| -8.51 | -7.68 | 1 |

Random forest

→ Titanic Dataset

| | pclass | sex | age | fare | survived |
|---|---|---|---|---|---|
| 0 | 1 | 1 | 29 | 211.33 | 1 |
| 1 | 1 | 0 | 0.91 | 151.55 | 1 |
| 2 | 1 | 1 | 2 | 151.55 | 0 |

→ Accuracy = 0.8129

Report

| | precision | recall | f1-score | support |
|---|---|---|---|---|
| 0 | 0.79 | 0.9 | 0.84 | 144 |
| 1 | 0.85 | 0.71 | 0.77 | 118 |

Code:

import kagglehub import pandas as

pd import numpy as np import

matplotlib.pyplot as plt from

sklearn.cluster import KMeans

path =

```
kagglehub.dataset_download("abda
llahwagih/mall-customers-
segmentation") print("Path to
dataset files:", path) import os
print(os.listdir(path)) df =
pd.read_csv(path +
"/Mall_Customers.csv") df.head()
df.info() df.describe()
X = df[['Annual Income (k$)', 'Spending Score (1-
100)']] wcss = [] for i in range(1, 11):
    kmeans = KMeans(n_clusters=i, random_state=42)
kmeans.fit(X)     wcss.append(kmeans.inertia_)
plt.plot(range(1,11), wcss) plt.title('Elbow Method')
plt.xlabel('Number of clusters') plt.ylabel('WCSS') plt.show()
kmeans = KMeans(n_clusters=5, random_state=42) y_kmeans
= kmeans.fit_predict(X) df['Cluster'] = y_kmeans df.head()
plt.scatter(X.iloc[:,0], X.iloc[:,1], c=y_kmeans,
cmap='rainbow') plt.scatter(kmeans.cluster_centers_[:,0],
kmeans.cluster_centers_[:,1], s=200, c='black')
plt.xlabel('Annual Income') plt.ylabel('Spending Score')
plt.title('Customer Segments') plt.show()
```

**Program 11**

Implement Dimensionality reduction using Principal Component Analysis (PCA) method.

Screenshot:

Date
Page

# Lab-10

→ Implement dimensionality reduction using Principle Component Analysis (PCA) method.

- PCA is a very popular dimensionality reduction technique. It easily transforms a set of possibly correlated variables into a smaller set of uncorrelated variables also known as principal components.

→ Steps:

i) Step 1: Data Loading

→ Loading that dataset that contains highly correlated features.

o/p: shape of data = (1000, 10)

ii) Step 2: Standardization

→ PCA is sensitive to the relative scales of the original features & features with larger variance will dominate the principal components if not scaled.

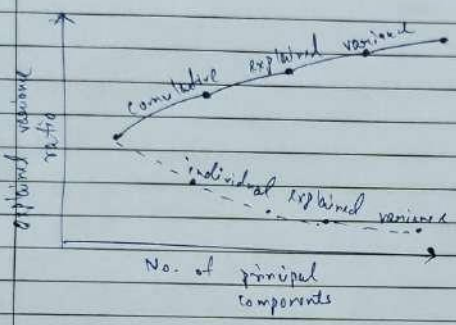
iii) Step 3: Applying PCA

- after standardizing the data, we can apply the PCA algorithm.
We will use PCA from sklearn.decomposition

Date __/__/__
Page _____

iv) Step 4: Modelling

- Choose a model, and then split the dataset and train the model on it.




→ Interpreting results:

- As the individual explained variance drops off, and the cumulative curve rises steeply initially, then flattens out. So around 7-8 components are essential and they retain about 95% of the original information.

∴ 10 features $\xrightarrow[\text{to}]{\text{reduced}}$ 7-8 features

15.0v

Code:

import numpy as np import pandas

as pd from sklearn.datasets import

load_iris data = load_iris() X =

data.data

```
y = data.target

X_meaned = X - np.mean(X, axis=0) cov_matrix =

np.cov(X_meaned, rowvar=False) eigenvalues,

eigenvectors = np.linalg.eigh(cov_matrix)

sorted_index = np.argsort(eigenvalues)[::-1]

sorted_eigenvalues = eigenvalues[sorted_index]

sorted_eigenvectors = eigenvectors[:, sorted_index] k

= 2   eigenvector_subset = sorted_eigenvectors[:, 0:k]

X_reduced = np.dot(X_meaned, eigenvector_subset)

print("Original shape:",X.shape) print("Reduced

shape:", X_reduced.shape)
```